

PHW251 Problem Set 3

Answer Key

Render this file from the `problem_set_3` folder so the relative paths to `../data` and `table_replicate.png` resolve.

Question 1

Create a data frame with three columns: ID (1, 2, 3), NAME ("Pam", "Jim", "Dwight"), and AGE (40, NA, 48).

```
office_df <- data.frame(  
  ID = c(1, 2, 3),  
  NAME = c("Pam", "Jim", "Dwight"),  
  AGE = c(40, NA, 48)  
)
```

```
office_df
```

	ID	NAME	AGE
1	1	Pam	40
2	2	Jim	NA
3	3	Dwight	48

Question 2

Find the length, typeof, and class of the data frame from Question 1.

```
length(office_df)
```

```
[1] 3
```

```
typeof(office_df)
```

```
[1] "list"
```

```
class(office_df)
```

```
[1] "data.frame"
```

A data frame is a list of columns, so `length()` is 3 (the number of columns), `typeof()` is "list", and `class()` is "data.frame".

Question 3

Create a data frame and a tibble that match `table_replicate.png`: columns `data_id`, `gender`, and `temperature`.

```
library(tibble)

covid_df <- data.frame(
  data_id = c(101, 102, 103, 104, 105),
  gender = c("female", "male", "non-binary", "male", NA),
  temperature = c(98, 97.3, 101.1, 97.5, 99.6)
)

covid_tib <- tibble(
  data_id = c(101, 102, 103, 104, 105),
  gender = c("female", "male", "non-binary", "male", NA),
  temperature = c(98, 97.3, 101.1, 97.5, 99.6)
)

covid_df
```

	data_id	gender	temperature
1	101	female	98.0
2	102	male	97.3
3	103	non-binary	101.1
4	104	male	97.5
5	105	<NA>	99.6

```
covid_tib
```

```
# A tibble: 5 x 3
  data_id gender    temperature
  <dbl> <chr>      <dbl>
1     101 female         98
2     102 male         97.3
3     103 non-binary    101.
4     104 male         97.5
5     105 <NA>         99.6
```

Question 4

What are the key differences between data frames and tibbles? Why are tibbles preferable?

Data frames and tibbles both store rectangular data, but tibbles change a few defaults that make them easier to work with:

- Tibbles do not use row names.
- Tibbles keep column names as you write them, including names with spaces or names that start with a number. Data frames will alter some of those names.
- A length-1 vector in a tibble is recycled to every row. You do not have to repeat a constant yourself.
- Inside `tibble()`, you can refer to a column you are creating in the same call.
- Printing a tibble shows the column types and only the first few rows, instead of dumping the whole table.
- Subsetting a tibble with `[, ]` still returns a tibble. The same subset of a data frame can drop down to a vector.

Tibbles are preferable because that behavior is more predictable, the printout is easier to read, and the rest of the tidyverse is built to work with them.

Question 5

Add a `results` column to the tibble from Question 3.

- 101 = NEGATIVE
- 102 = POSITIVE
- 103 = NEGATIVE
- 104 = NEGATIVE
- 105 = NEGATIVE

```
covid_tib$results <- c("NEGATIVE", "POSITIVE", "NEGATIVE",  
                      "NEGATIVE", "NEGATIVE")  
covid_tib
```

```
# A tibble: 5 x 4  
  data_id gender      temperature results  
    <dbl> <chr>          <dbl> <chr>  
1     101 female           98  NEGATIVE  
2     102 male           97.3 POSITIVE  
3     103 non-binary    101.  NEGATIVE  
4     104 male           97.5 NEGATIVE  
5     105 <NA>          99.6 NEGATIVE
```

Question 6

Subject 102's temperature is 98.3, not 97.3.

```
covid_tib$temperature[covid_tib$data_id == 102] <- 98.3
covid_tib
```

```
# A tibble: 5 x 4
  data_id gender      temperature results
  <dbl> <chr>          <dbl> <chr>
1    101 female             98  NEGATIVE
2    102 male             98.3 POSITIVE
3    103 non-binary        101.  NEGATIVE
4    104 male             97.5 NEGATIVE
5    105 <NA>             99.6 NEGATIVE
```

Question 7

Load `stds-by-disease-county-year-sex.csv` from the data folder. The first three rows are not part of the table, so skip them.

```
stds <- read.csv(file = "../data/stds-by-disease-county-year-sex.csv",  
                 skip = 3)  
head(stds)
```

	Disease	County	Year	Sex	Cases	Population
1	Chlamydia	California	2001	Female	75941	17339700
2	Chlamydia	California	2001	Male	24885	17173042
3	Chlamydia	California	2001	Total	101590	34512742
4	Chlamydia	California	2002	Female	81583	17554666
5	Chlamydia	California	2002	Male	28521	17383624
6	Chlamydia	California	2002	Total	110759	34938290

Question 8

How many rows?

```
nrow(stds)
```

```
[1] 9558
```

How many columns?

```
ncol(stds)
```

```
[1] 6
```

What are the column names?

```
colnames(stds)
```

```
[1] "Disease"  "County"   "Year"     "Sex"      "Cases"
[6] "Population"
```

What are the column types?

```
sapply(stds, class)
```

```
      Disease      County      Year      Sex      Cases      Population
"character" "character" "integer" "character" "integer" "integer"
```

There are 9558 rows and 6 columns: Disease, County, Year, Sex, Cases, and Population. Year, Cases, and Population are integers. The other three columns are character.

Question 9

Keep the rows for 2015-2018 with `which()`. The subset should be 2124 rows by 6 columns.

```
stds_recent <- stds[which(stds$Year >= 2015 & stds$Year <= 2018), ]  
dim(stds_recent)
```

```
[1] 2124    6
```

Question 10

Export the 2015-2018 subset as a csv in the data folder, then read it back.

```
write.csv(stds_recent,  
         file = "../data/stds_2015_2018.csv",  
         row.names = FALSE)  
  
stds_check <- read.csv(file = "../data/stds_2015_2018.csv")  
head(stds_check)
```

	Disease	County	Year	Sex	Cases	Population
1	Chlamydia	California	2015	Female	121749	19634752
2	Chlamydia	California	2015	Male	67694	19441376
3	Chlamydia	California	2015	Total	189747	39076128
4	Chlamydia	California	2016	Female	123924	19758238
5	Chlamydia	California	2016	Male	73708	19570099
6	Chlamydia	California	2016	Total	198245	39328337

```
dim(stds_check)
```

```
[1] 2124    6
```

Question 11

`unique()` on `County` should return 59 counties. Then subset to one county.

```
counties <- unique(stds$County)
counties
```

```
[1] "California"      "Alameda"         "Alpine"           "Amador"
[5] "Butte"           "Calaveras"       "Colusa"           "Contra Costa"
[9] "Del Norte"       "El Dorado"       "Fresno"           "Glenn"
[13] "Humboldt"        "Imperial"        "Inyo"             "Kern"
[17] "Kings"           "Lake"            "Lassen"           "Los Angeles"
[21] "Madera"          "Marin"           "Mariposa"         "Mendocino"
[25] "Merced"          "Modoc"           "Mono"             "Monterey"
[29] "Napa"            "Nevada"          "Orange"           "Placer"
[33] "Plumas"          "Riverside"       "Sacramento"       "San Benito"
[37] "San Bernardino" "San Diego"       "San Francisco"    "San Joaquin"
[41] "San Luis Obispo" "San Mateo"       "Santa Barbara"    "Santa Clara"
[45] "Santa Cruz"      "Shasta"          "Sierra"           "Siskiyou"
[49] "Solano"          "Sonoma"          "Stanislaus"       "Sutter"
[53] "Tehama"          "Trinity"         "Tulare"           "Tuolumne"
[57] "Ventura"         "Yolo"            "Yuba"
```

```
length(counties)
```

```
[1] 59
```

```
alameda <- stds[stds$County == "Alameda", ]
head(alameda)
```

	Disease	County	Year	Sex	Cases	Population
55	Chlamydia	Alameda	2001	Female	3691	746596
56	Chlamydia	Alameda	2001	Male	1126	718968
57	Chlamydia	Alameda	2001	Total	4861	1465564
58	Chlamydia	Alameda	2002	Female	3729	747987
59	Chlamydia	Alameda	2002	Male	1126	720481
60	Chlamydia	Alameda	2002	Total	4870	1468468

Question 12

In the county subset, add `rate = (Cases / Population) * 100,000`.

```
alameda$rate <- (alameda$Cases / alameda$Population) * 100000  
head(alameda)
```

	Disease	County	Year	Sex	Cases	Population	rate
55	Chlamydia	Alameda	2001	Female	3691	746596	494.3771
56	Chlamydia	Alameda	2001	Male	1126	718968	156.6134
57	Chlamydia	Alameda	2001	Total	4861	1465564	331.6812
58	Chlamydia	Alameda	2002	Female	3729	747987	498.5381
59	Chlamydia	Alameda	2002	Male	1126	720481	156.2845
60	Chlamydia	Alameda	2002	Total	4870	1468468	331.6381

Question 13

Reply to the classmate whose import code is not a repeatable example. Include at least three specific suggestions.

Three changes that would make this a repeatable example:

1. Delete `rm(list = ls())`. Clearing the whole environment is not part of the example, and it can wipe objects the other person needs in order to see the problem.
2. Load the package before using `read_csv()`. A repeatable example has to include `library(readr)` or `library(tidyverse)`, because another computer will not already have that function attached.
3. Do not depend on a local file the reader does not have. Paste a few rows with `dput()`, or use a small built-in data set, and include the exact error message. The arguments `header` and `row.names` also belong to `read.csv()`, not `read_csv()`, so the call should use the arguments for the function that is actually being run.